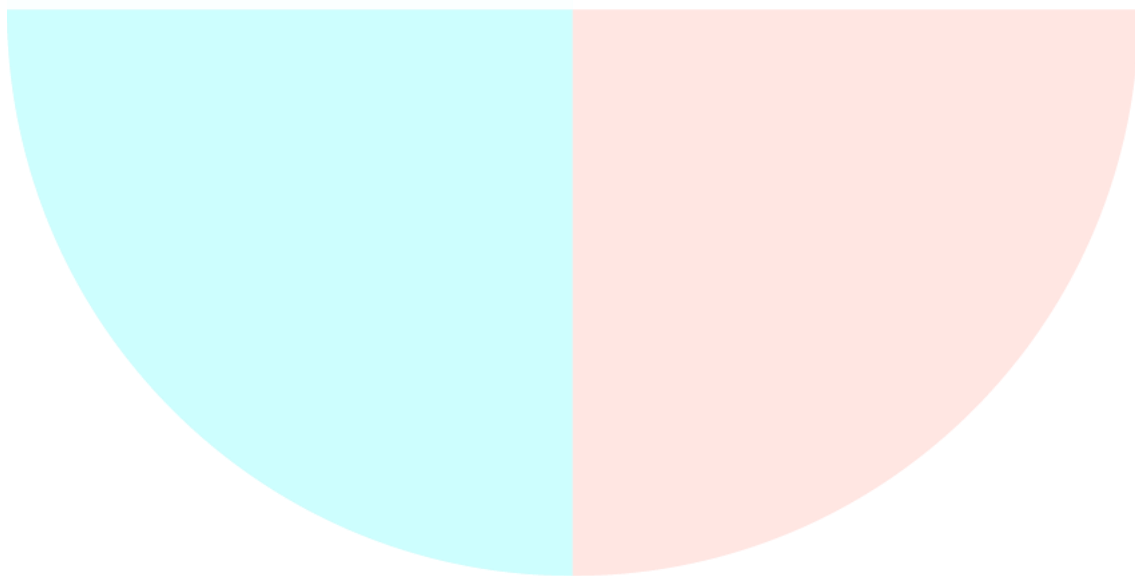


Bloque 3 - Tema 2

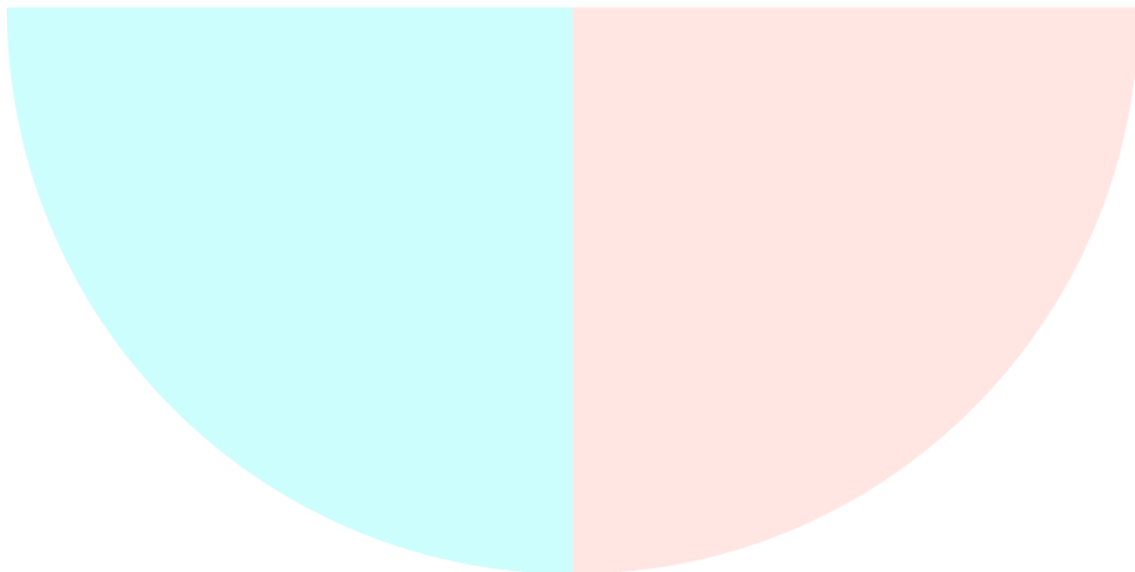
DISEÑO DE BASES DE DATOS. DISEÑO LÓGICO Y FÍSICO. EL MODELO LÓGICO RELACIONAL. NORMALIZACIÓN



PREPARACIÓN OPOSICIONES
TÉCNICOS AUXILIARES DE INFORMÁTICA

ÍNDICE

ÍNDICE	2
1. INTRODUCCIÓN	3
2. DISEÑO DE BASES DE DATOS	5
3. DISEÑO LÓGICO	9
1. Ajustes del modelo conceptual.....	9
2. Obtención del modelo lógico a partir del conceptual	11
4. DISEÑO FÍSICO.....	17
1. Traducir el esquema lógico para el SGBD específico.....	17
2. Diseñar la representación física	18
3. Diseñar los mecanismos de seguridad	21
4. Monitorizar y afinar el sistema	21
5. EL MODELO LÓGICO RELACIONAL.....	22
1. Conceptos fundamentales del modelo relacional	22
2. Reglas de integridad	23
3. Lenguajes relaciones	25
6. NORMALIZACIÓN	30
1. Primera Forma normal (1FN)	31
2. Segunda Forma normal (2FN)	32
3. Tercera Forma normal (3FN).....	33
4. Forma normal de Boyce-Codd (FNBC)	33
5. Cuarta Forma normal (4FN)	34
6. Quinta Forma normal (5FN).....	35



1. INTRODUCCIÓN

El diseño y construcción de una base de datos es un proceso de abstracción y modelización de un problema real, conforme a unas reglas y formalismos determinados, con el objetivo de conseguir una representación del mismo tratable por el ordenador.

Esto implica partir de un problema claramente determinado, definir cuáles son las entidades que participan en el mismo y sus atributos (fase de abstracción), construir un modelo y, por último, transformarlo para su implementación mediante algún lenguaje o herramienta concretos.

El problema de la modelización de datos comienza con el estudio y captura de requisitos en la etapa conocida como Análisis. Con estos requisitos es posible representar el problema desde un punto de vista conceptual, obteniendo el **modelo conceptual** a través de la técnica denominada Modelo de Entidad-Relación Extendido.

Una vez validado es posible transformarlo a un **modelo lógico específico** (jerárquico, red, relacional u orientado a objetos) dentro de la etapa conocida como Diseño.

Finalmente, se hace necesario implantar este modelo lógico en alguna de las soluciones concretas de SGBD (**modelo físico**) dentro de la etapa conocida como Construcción del sistema.



El **diseño LÓGICO** es el proceso de construir un esquema de la información que utiliza una organización teniendo en cuenta un modelo lógico independiente del SGBD que se vaya a utilizar y de cualquier otra consideración física.

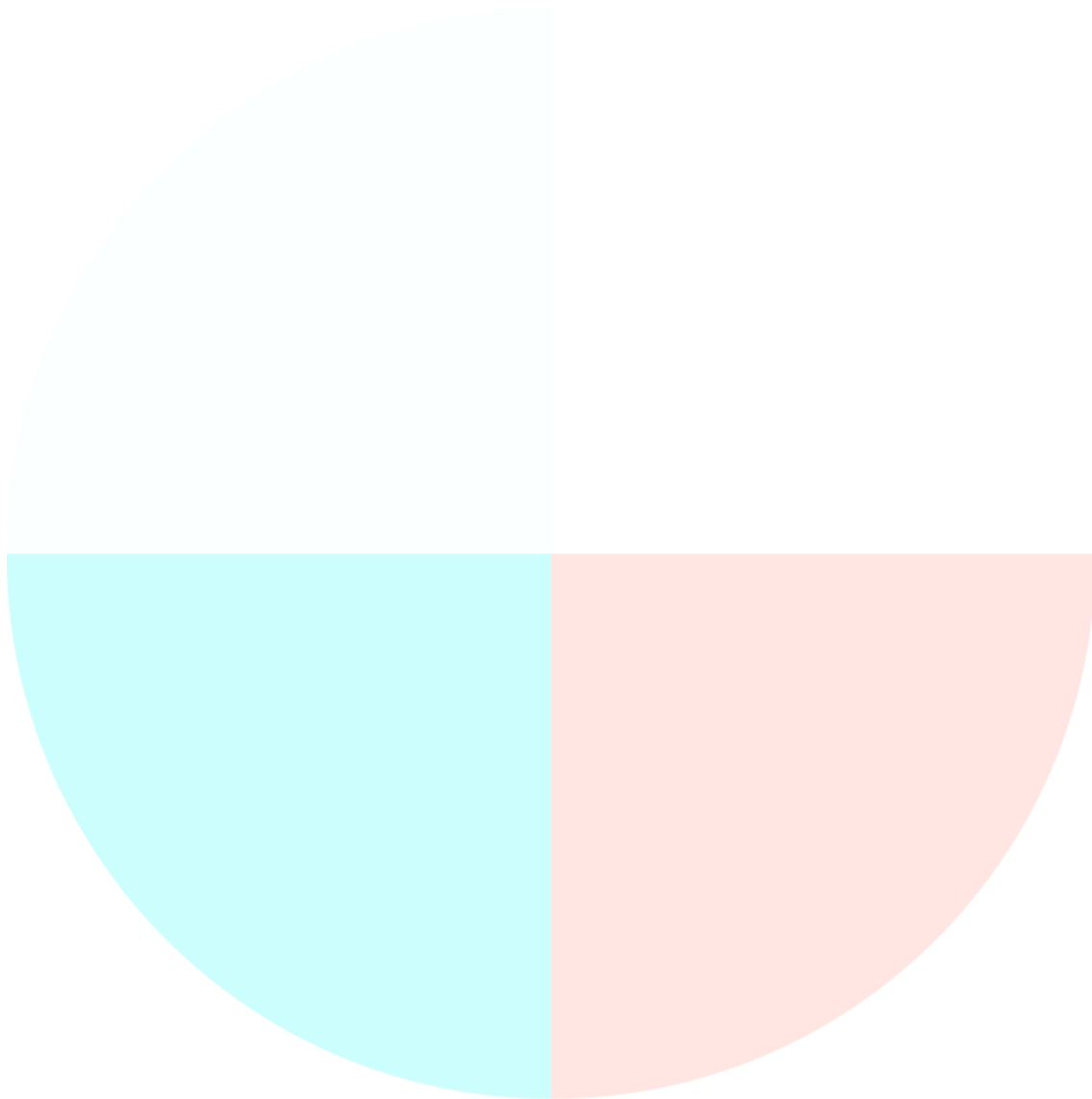
En esta etapa, se transforma el esquema conceptual en un esquema lógico que utilizará las estructuras de datos del modelo lógico en el que se basa el SGBD (jerárquico, red, relacional, orientado a objetos). Conforme se va desarrollando el esquema lógico, éste se va probando y validando con los requisitos de usuario.

La **normalización** es una técnica que se utiliza para comprobar la validez de los esquemas lógicos basados en el modelo relacional, ya que asegura que las relaciones (tablas) obtenidas no tienen datos redundantes, obteniéndose el **modelo lógico de datos normalizado**.

Tanto el diseño conceptual, como el diseño lógico, son procesos iterativos, tienen un punto de inicio y se van refinando continuamente. Ambos se deben ver como un proceso de aprendizaje en el que el diseñador va comprendiendo el funcionamiento de la organización y

el significado de los datos que maneja. Por tanto, el diseño conceptual y el diseño lógico son etapas clave para conseguir un sistema que funcione correctamente.

El **diseño FÍSICO** es el proceso de descripción de la implementación de la base de datos en memoria secundaria: estructuras de almacenamiento y métodos de acceso que garanticen un acceso eficiente a los datos. En general, el propósito del diseño físico es describir cómo se va a implementar físicamente el esquema lógico obtenido en la fase anterior.



2. DISEÑO DE BASES DE DATOS

Las bases de datos son parte integral de cualquier sistema de información. Por tanto, una de las características que deben presentar es la capacidad de adaptación a cambios en el entorno. Estos cambios, inevitables en cualquier organización, pueden ser físicos (cambios en el hardware, en el formato de los ficheros, etc.) o lógicos (cambios en los programas, en el lenguaje de programación, etc.).

No es deseable que un cambio en el formato de archivo de los datos obligue a rehacer la aplicación que accede a los mismos. Es necesario, pues, independizar la estructura lógica de los datos de la forma en que estos se guardan físicamente. Así, cualquier cambio en uno de los dos niveles será transparente para el otro, facilitando la tarea de mantenimiento, tanto de las aplicaciones como de las bases de datos.

Esta independencia entre el modelo lógico de los datos y su estructura física es la que proporciona la **arquitectura en 3 niveles del Instituto Nacional de Estandarización Americano (ANSI)**. Según este organismo la independencia de los datos es la capacidad de un sistema de gestión de base de datos para permitir que las referencias a los datos a través de los programas estén aisladas de los posibles cambios y diferentes usos que el entorno pueda propiciar, como pueden ser: la forma de almacenamiento, el modo de compartición con otros programas o el modo de organización para mejorar el rendimiento del sistema de base de datos.

Independencia de los datos

La arquitectura de tres niveles es útil para explicar el concepto de **independencia de datos** que podemos definir como la capacidad para modificar el esquema en un nivel del sistema sin tener que modificar el esquema del nivel inmediato superior.

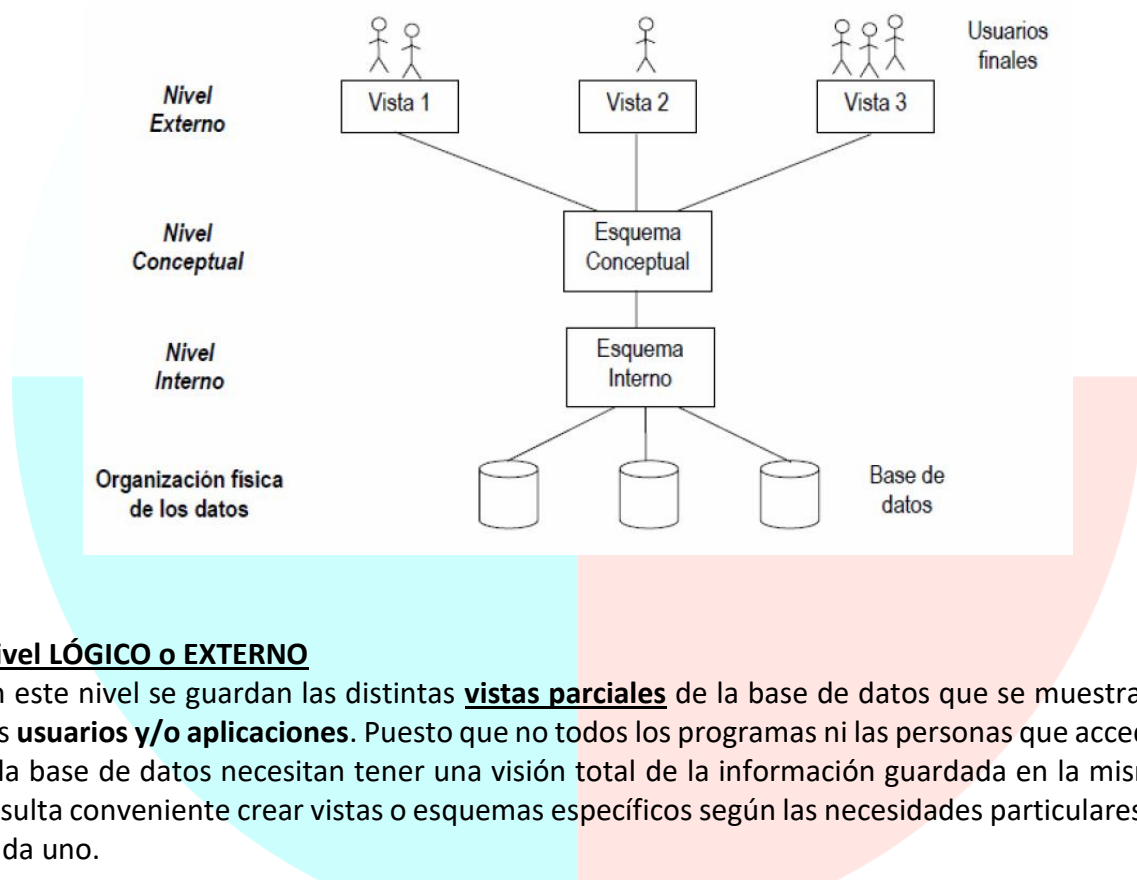
Se pueden definir dos tipos de independencia de datos:

- La **independencia LÓGICA** es la capacidad de modificar el esquema conceptual sin tener que alterar los esquemas externos ni los programas de aplicación. Se puede modificar el esquema conceptual para ampliar la base de datos o para reducirla. Si, por ejemplo, se reduce la base de datos eliminando una entidad, los esquemas externos que no se refieran a ella no deberán verse afectados.
- La **independencia FÍSICA** es la capacidad de modificar el esquema interno sin tener que alterar el esquema conceptual (o los externos). Por ejemplo, puede ser necesario reorganizar ciertos ficheros físicos con el fin de mejorar el rendimiento de las operaciones de consulta o de actualización de datos. Dado que la independencia física se refiere sólo a la separación entre las aplicaciones y las estructuras físicas de almacenamiento, es más fácil de conseguir que la independencia lógica.

Para conseguir este ideal de independencia, ANSI propone que las bases de datos se construyan siguiendo un modelo o arquitectura de 3 niveles:

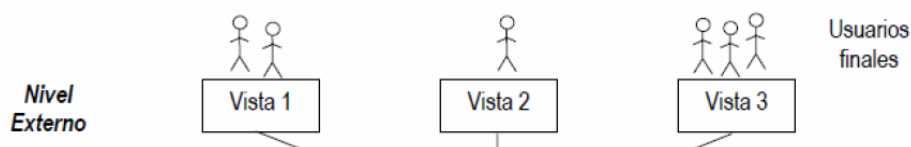
- Nivel externo o lógico.
- Nivel conceptual.
- Nivel físico o interno.

Estos 3 niveles están organizados de forma jerárquica, siendo el nivel físico el más cercano a la máquina y el nivel externo aquél con el que interactúa el usuario. Esta arquitectura por capas sirve para aislar al usuario de las complejidades del modelo de datos y de almacenamiento físico de la base, ya que sólo se le permitirá ver la parte lógica de la base necesaria para su trabajo. Además, también aísla el modelo lógico de datos (aquí llamado conceptual) de la implementación específica que se realice del mismo, por lo que, en teoría, éste sería portable de unos gestores de bases de datos a otros. Todo ello dio lugar a la denominada **Arquitectura ANSI/X3/SPARC**.



Nivel LÓGICO o EXTERNO

En este nivel se guardan las distintas **vistas parciales** de la base de datos que se muestran a los **usuarios y/o aplicaciones**. Puesto que no todos los programas ni las personas que acceden a la base de datos necesitan tener una visión total de la información guardada en la misma, resulta conveniente crear vistas o esquemas específicos según las necesidades particulares de cada uno.



La existencia de este nivel aísla por completo a los usuarios y/o aplicaciones no sólo del aspecto físico de la base de datos, sino también de cualquier parte de la misma que no esté directamente relacionada con la tarea que deben desempeñar, aumentando así la independencia y la protección frente a cambios en otras áreas de la organización. Adicionalmente, esta capa también aumenta la seguridad de los datos, ya que, como

consecuencia de la creación de vistas parciales de la base de datos, los usuarios no disponen de acceso más que a partes seleccionadas de la misma, disminuyendo así la posibilidad de consultar, modificar o borrar datos privilegiados.

A partir del **nivel externo se crea el esquema externo** para cada base de datos en concreto. Los esquemas externos pueden ser múltiples para una misma base de datos y además se puede producir anidamiento entre ellos. El esquema externo queda representado por los datos que de forma efectiva pueda acceder cada usuario y/o aplicación, más los permisos de acceso efectivos sobre los mismos.

Nivel CONCEPTUAL

Este modelo pretende reflejar la **estructura y relaciones existentes entre los datos** del mundo real que se van a guardar en la base de datos, aislando entre sí los niveles externo (vista del usuario) e interno (vista de la máquina).

Para construir el esquema es necesario definir el universo del discurso o, lo que es lo mismo, acotar aquella parte del mundo real que se quiere modelar, incluyendo todos los elementos o características relevantes y excluyendo aquellas que pese a existir en el problema original no son relevantes. Para ello, se utiliza típicamente la técnica de Modelo Entidad-Relación Extendido.

A partir del **modelo conceptual se obtiene el esquema conceptual** que se identifica mediante las estructuras de datos, relaciones y restricciones.

Dentro de este nivel se incluye también la manera de obtener una adecuación del mismo al entorno de implantación elegido, es decir, el modelo lógico.

El **modelo lógico de datos** se define como un conjunto de conceptos, reglas y convenciones que permiten describir un modelo conceptual. Los modelos de datos comúnmente utilizados son:

- **Modelo JERÁRQUICO**: presenta una estructura en árbol donde nodos y ramas siguen una relación del tipo 1:N.
- **Modelo CODASYL (Conference on Data Systems Languages)**: estructura en red donde se establecen relaciones N:M. Es más flexible que el jerárquico.
- **Modelo RELACIONAL**: presenta estructuras de la teoría matemática de conjuntos (álgebra relacional) y/o de la lógica de predicados (cálculo relacional). Permite el procesamiento de conjuntos de datos y no simples registros como en los anteriores. Se caracteriza por disponer los datos organizados en tablas (relaciones) que cumplen ciertas restricciones.
- **Modelo ORIENTADO A OBJETOS**.

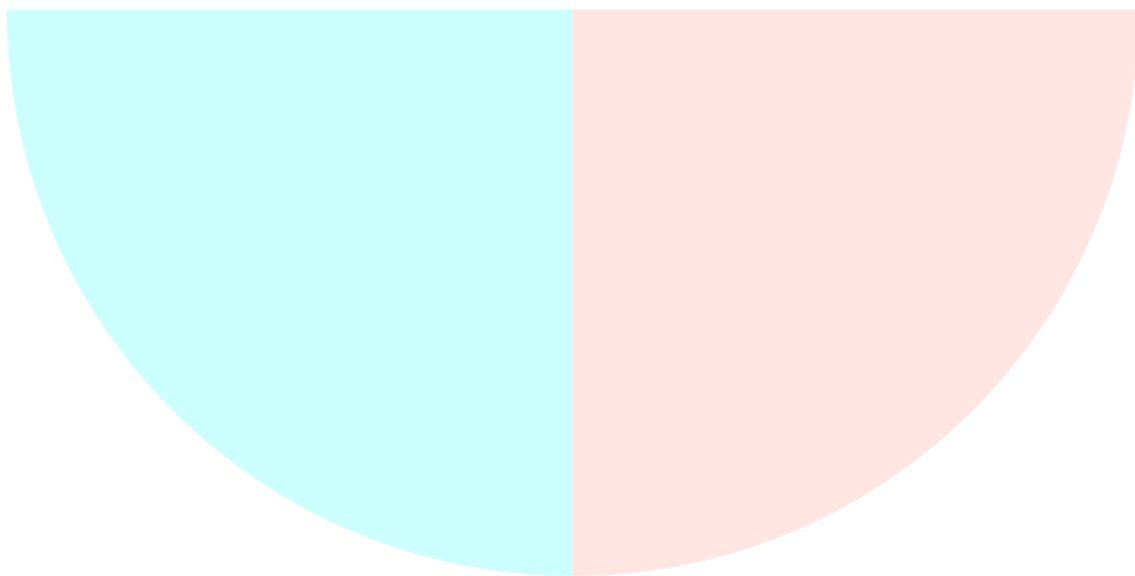
Cada uno de estos modelos tiene sus características únicas que los hacen más adecuados para modelar unos problemas u otros, así mismo, el modelo elegido va a condicionar en gran medida los lenguajes de datos utilizados, ya que la propia estructura del modelo llega a imponer determinadas formas de acceder a los datos.

Nivel INTERNO o FÍSICO

En este nivel se especifica **qué, cómo y dónde se van a almacenar los datos físicamente**. Se ocupa de tratar con el sistema operativo, con el sistema de ficheros, con los dispositivos de entrada/ salida y, en general, con todos aquellos aspectos de bajo nivel necesarios para almacenar efectivamente la información en el ordenador. El contenido de este nivel depende por completo de la combinación hardware/software que se emplee en cada instalación.

Del **nivel físico se deriva el esquema interno**, que contiene las definiciones de los registros guardados, los métodos de representación, los campos de datos y los índices. Hay un solo esquema interno por base de datos.

Como resumen de esta arquitectura, su propósito principal es que el Esquema Conceptual sea una descripción estable de la organización e independiente de las “vistas” y de la forma de almacenamiento de los datos. Debido a esta independencia de niveles, las Bases de Datos pueden ser flexibles y adaptables a los cambios.



3. DISEÑO LÓGICO

A partir del esquema conceptual se elabora el modelo lógico. Este modelo debe coincidir con el modelo datos soportado por el SGBD que se vaya a utilizar. Es nuestro caso el modelo de datos es el modelo relacional.

Las técnicas de modelado conceptual son diferentes de las técnicas de modelado lógico, por lo que habrá que convertir cada elemento presente en el modelo conceptual en elementos expresables en la técnica de modelado lógico que hayamos seleccionado. Para ello se realizan una serie de transformaciones y el proceso de normalización.

1. Ajustes del modelo conceptual

ELIMINACIÓN DE ATRIBUTOS COMPUESTOS

Estos atributos se caracterizan porque se pueden descomponer en descomponer en una enumeración de atributos simples que expresan de forma más precisa la información a representar.

Ejemplo: el atributo “Código_Cuenta_Bancaria” de la entidad “Cuentas Banco” se podría descomponer:

- Código_Banco.
- Código_Sucursal.
- Número_de_cuenta.

El resultado es una información más precisa y al mismo tiempo más breve al haberse eliminado el Código de Control al ser posible calcularlo a partir de los demás campos.

Esta descomposición de los atributos compuestos en otros simples se realiza, fundamentalmente, para facilitar el acceso a la información y su posterior tratamiento.

Como resultado de esta eliminación se obtiene un Diagrama Entidad-Relación en el cual **todas las entidades tienen atributos simples**.

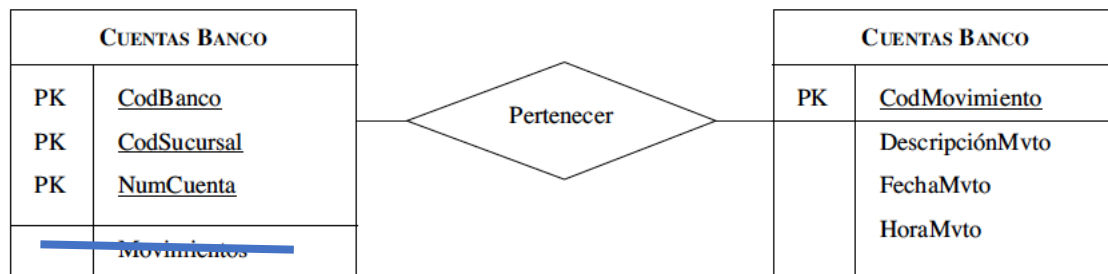
ELIMINACIÓN DE ATRIBUTOS MULTIVALORADOS O MULTIVALUADOS

Los atributos multivalorados son aquellos que pueden tomar más de un valor para una misma ocurrencia de una entidad.

CUENTAS BANCO	
PK	<u>CodBanco</u>
PK	<u>CodSucursal</u>
PK	<u>NumCuenta</u>
	Movimientos

Se observa que el atributo Movimientos puede tomar más de un valor, ya que una Cuenta bancaria normalmente tiene más de un movimiento. Este tipo de atributos no son válidos en el modelo relacional, por lo que es necesario eliminarlos.

La solución al problema consiste en crear una nueva entidad que represente al atributo multivalorado y una relación entre esta entidad y la original. En nuestro ejemplo, esto se haría de la siguiente manera:

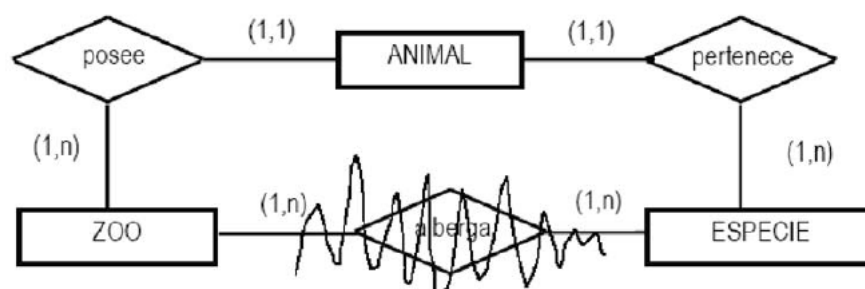


Por lo tanto, un movimiento pertenece a una cuenta y una cuenta puede tener múltiples movimientos.

Como resultado de esta eliminación se obtiene un Diagrama Entidad-Relación en el cual todas las entidades tienen atributos univaluados.

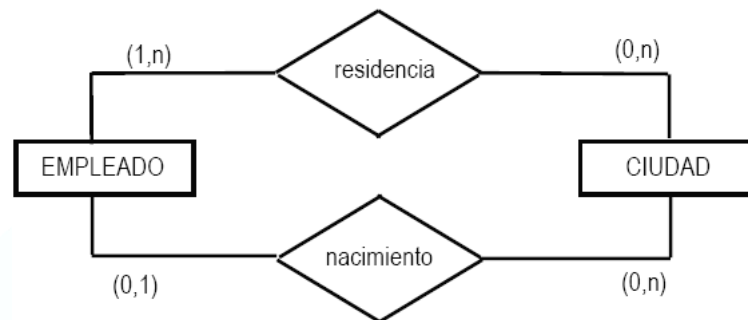
ELIMINACIÓN DE RELACIONES REDUNDANTES

Una relación es redundante cuando se puede obtener la misma información que aporta mediante otras relaciones. El hecho de que haya dos caminos diferentes entre dos entidades no implica que uno de los caminos corresponda a una relación redundante, eso dependerá de la carga semántica que aporte cada una de las relaciones representadas.



Se observa que la relación “Alberga” es innecesaria puesto que, en principio, podríamos conocer todas las especies de animales existentes en un zoo a través de la relación entre la entidad “Animal” y “Especies” al existir también una relación entre “Zoo” y “Animal”.

Sin embargo, en el siguiente ejemplo no sería normal que se eliminara alguna de las relaciones, aun cuando estén vinculando a las mismas entidades, ya que la carga semántica de cada una de ellas es diferente:



2. Obtención del modelo lógico a partir del conceptual

TRANSFORMACIÓN DE DOMINIOS

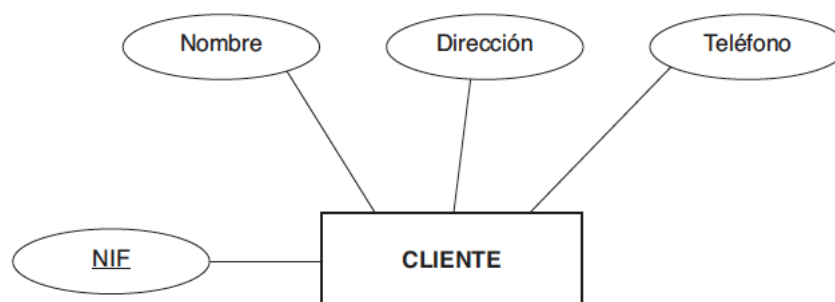
Un dominio del modelo conceptual se transforma en un dominio equivalente del modelo lógico.

Para ello, se emplea la sentencia CREATE DOMAIN. En otro caso, será necesario utilizar los tipos primitivos más afines con el dominio representable y delimitar sus elementos mediante restricciones de usuario asociadas a la tabla a la que pertenezca el atributo con tal dominio.

TRANSFORMACIÓN DE ENTIDADES

Cada entidad del modelo conceptual se transforma en una relación o tabla con estructura relacional. La clave primaria de la entidad pasa a ser la clave primaria de la relación. Los atributos que forman parte de la clave no podrán tomar el valor nulo.

Ejemplo: Dado el siguiente diagrama conceptual:



Darí­a como resultado esta relación:

CLIENTE(NIF (PK), Nombre, Dirección, Teléfono)

TRANSFORMACIÓN DE ATRIBUTOS

Cada atributo se transforma en una columna de la tabla en la que se transformó la entidad a la que pertenece. El identificador único se convierte en clave primaria.

- **Claves Primarias o Identificadores:** las claves primarias o identificadores de la entidad pasan a ser claves primarias en la relación resultantes (PRIMARY KEY).
- **Claves candidatas (o alternativas):** se transforman como atributos convencionales, pero en su implementación deberá añadirse la restricción UNIQUE.
- **Atributos convencionales:** se transforman en campos de la relación con el dominio que tuvieran asignado.
- **Atributos compuestos y multivalorados:** se transforman tal y como se indicaba más arriba.

TRANSFORMACIÓN DE RELACIONES

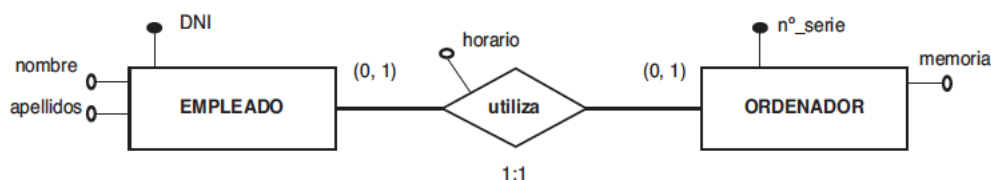
Relaciones 1:1

Como norma general, es un caso particular de las 1:N y por tanto se propaga la clave en las dos direcciones. Se debe analizar la situación, intentando recoger la mayor semántica posible, y evitar valores nulos.

1) Será necesario crear una nueva tabla si:

- Las cardinalidades mínimas son cero (ambas), esto evitará valores nulos en las claves ajenas y mantendrá la simetría natural (las entidades mantienen su independencia en tablas separadas)
- La relación tiene atributos propios.
- Se prevé que posteriormente puedan variarse las cardinalidades.

Ejemplo:



Se crearán tres tablas:

EMPLEADO(DNI (PK), Nombre, Apellido)

ORDENADOR(NumSerie (PK), Memoria)

UTILIZA(NumSerie+DNI (PK), horario)

- 2) Si una de las cardinalidades mínimas es cero y la otra no, será (1,1), conviene propagar la clave de esta última (la obligatoria) a la primera.
- 3) Si las dos entidades participan de forma completa, es decir, todas cardinalidades mínimas son 1:
 - a. Si las entidades tienen el mismo identificador se transforman en una única tabla formada por la concatenación de los atributos de los dos tipos de entidad.
 - b. En el caso de que tengan diferente identificador, cada entidad se transforma en una tabla y se puede propagar la clave de cualquiera de ellas a la tabla resultante de la otra, teniendo en cuenta, en este caso, los accesos más frecuentes y prioritarios a los datos de las tablas.

Relaciones 1:N

Según el caso que nos ocupe:

- 1) Relaciones entre **entidades fuertes**. Se utiliza el método de propagación de clave. En este caso, el identificador de la entidad con cardinalidad uno cede su clave a la de cardinalidad muchos. En esta última, pasa a ser una clave ajena.
- 2) Relaciones de **dependencia** (entidad fuerte-entidad débil):
 - a. Dependencia por existencia. Igualmente, se utiliza el método de propagación de clave (punto 1).
 - b. Dependencia por identificador. Se utiliza el método de propagación de clave, y además la clave primaria de la tabla con cardinalidad muchos estará formada por la concatenación de su propia clave más la que le cede la de cardinalidad uno.

Relaciones N:M y ternarias

Se crea una tabla como consecuencia de la relación que tendrá como clave primaria la concatenación de los identificadores de las entidades relacionadas. La condición de clave ajena se expresa mediante FOREIGN KEY.

Relaciones de agregación

Las relaciones de agregación se transforman del mismo modo que las 1:N.

Relaciones Reflexivas

Cuando se trata de relaciones reflexivas con correspondencia 1:N se transforma utilizando el método de propagación de clave, aunque, en este caso, al tratarse de la misma tabla, es necesario renombrar el nombre del identificador que se transfiere, ya que si no estaría repetido respecto del ya existente en la entidad de origen.

Transformación de relaciones exclusivas

Después de haber realizado la transformación según las relaciones 1:N, se debe tener en cuenta que si los identificadores propagados se han convertido en claves ajenas de la tabla originada por la entidad común a las relaciones, hay que comprobar que una y sólo una de esas claves es nula en cada ocurrencia. En otro caso, estas comprobaciones se deben hacer en las tablas resultantes de transformar las relaciones.

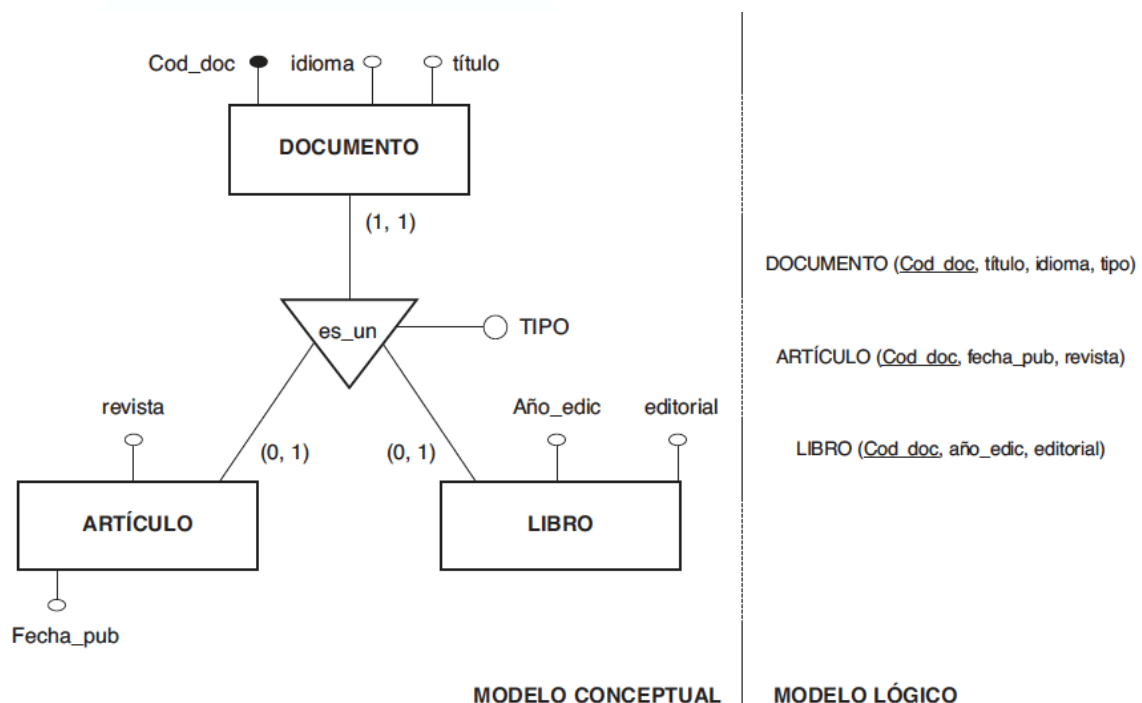
TRANSFORMACIÓN DE JERARQUÍAS

En el modelo lógico relacional no se dispone de instrumentos que permitan representar supertipos y subtipos. Se definen distintos métodos de transformación, dependiendo de los objetivos perseguidos:

- Información semántica representada en el modelo.
- Eficiencia de acceso a los datos.

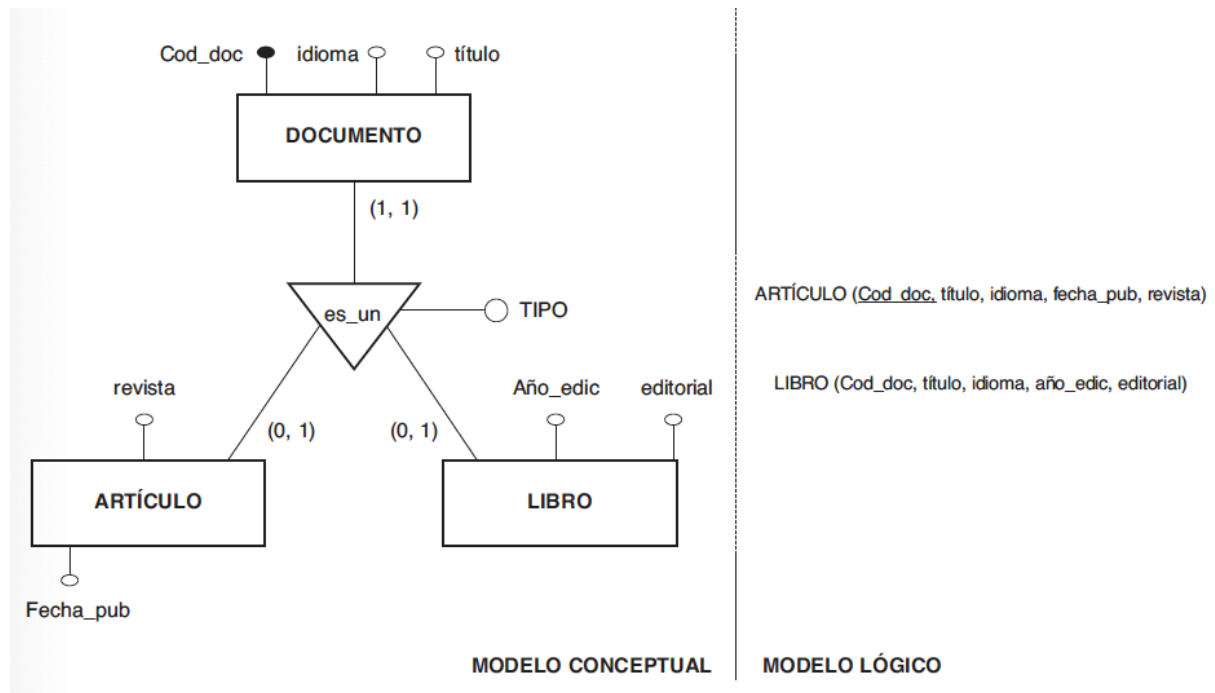
Opción 1. Consiste en crear una tabla para el supertipo que tenga de clave primaria el identificador y una tabla para cada uno de los subtipos que tengan el identificador del supertipo como clave ajena.

Esta solución es apropiada cuando los subtipos tienen muchos atributos distintos y se quieren conservar los atributos comunes en una tabla. También se deben definir las restricciones y aserciones adecuadas. Es la solución que mejor conserva la semántica.



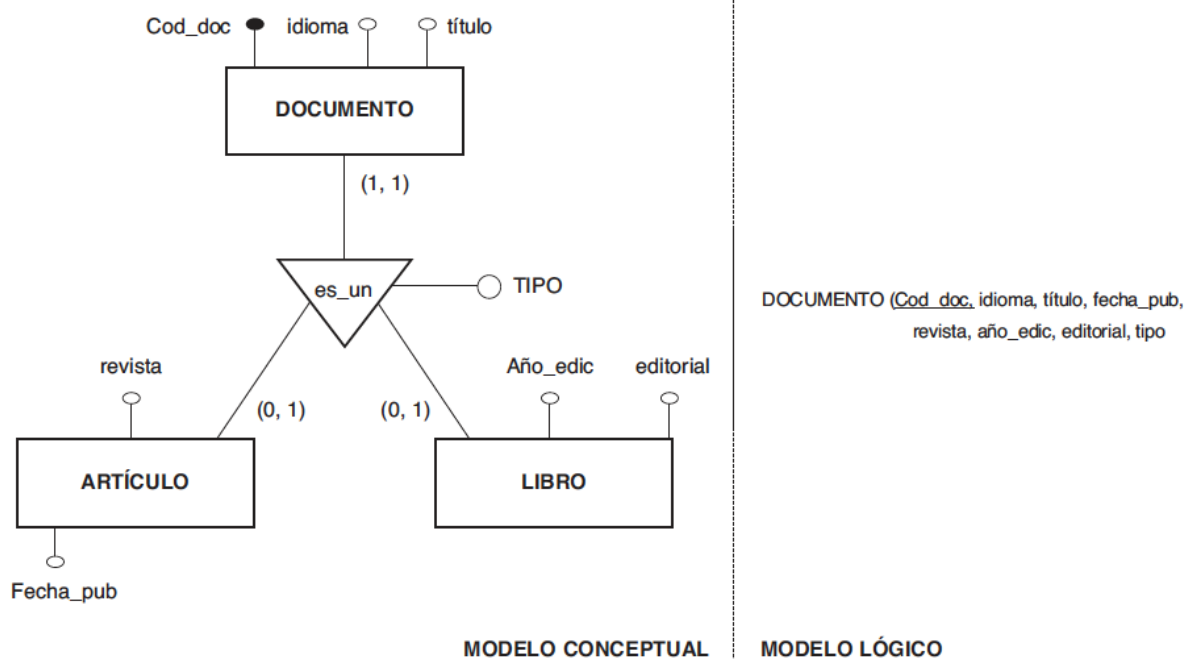
Opción 2. Se crea una tabla para cada subtipo, los atributos comunes aparecen en todos los subtipos y la clave primaria para cada tabla es el identificador del supertipo.

Esta opción mejora la eficiencia en los accesos a todos los atributos de un subtipo, sean los comunes al supertipo o los específicos.

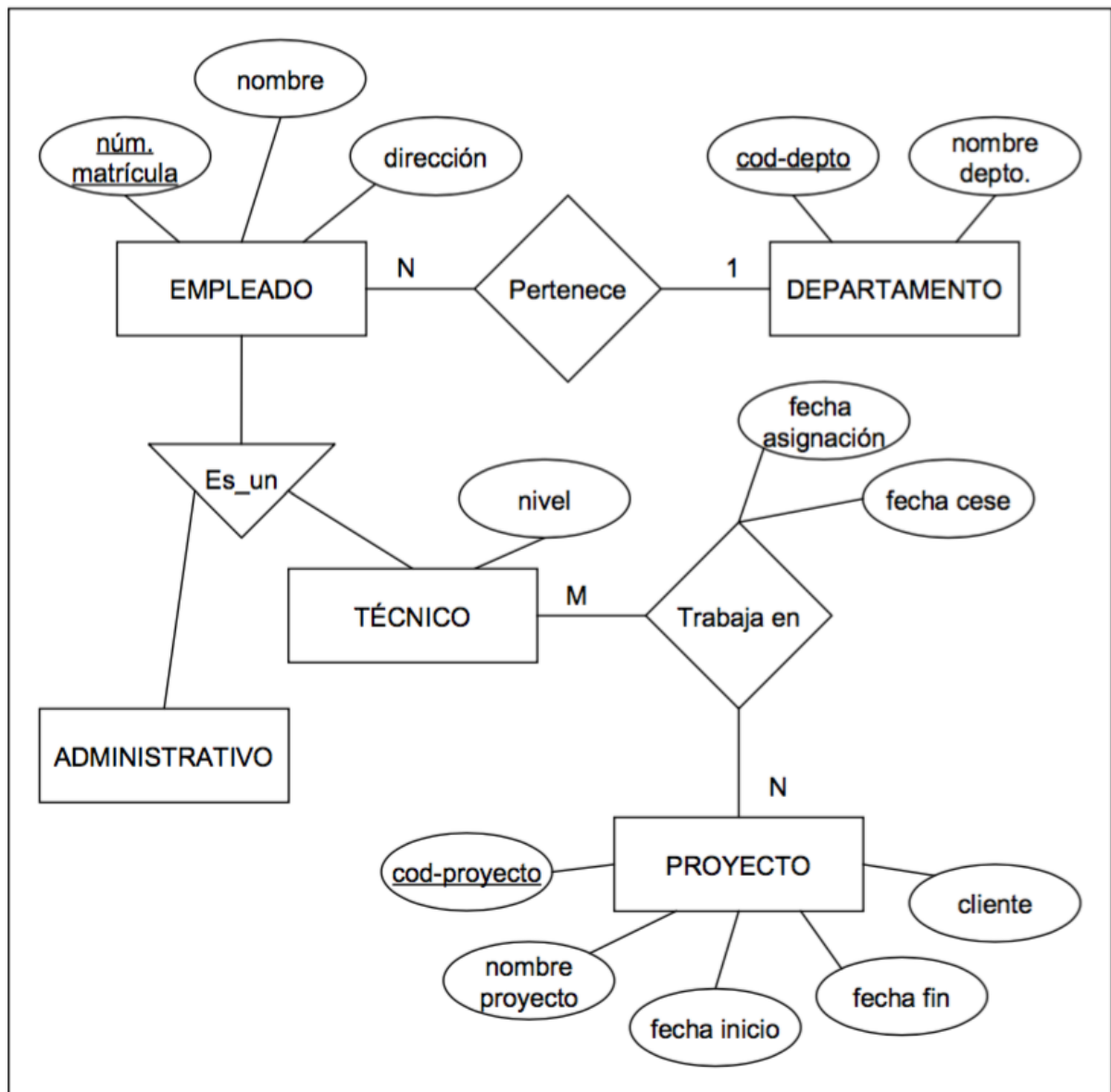


Opción 3. Agrupar en una tabla todos los atributos de la entidad supertipo y de los subtipos. La clave primaria de esta tabla es el identificador de la entidad. Se añade un atributo que indique a qué subtipo pertenece cada ocurrencia (el atributo discriminante de la jerarquía).

Esta solución puede aplicarse cuando los subtipos se diferencien en pocos atributos y las relaciones entre los subtipos y otras entidades sean las mismas. Para el caso de que la jerarquía sea total, el atributo discriminante no podrá tomar valor nulo (ya que toda ocurrencia pertenece a alguna de las entidades subtipo).



Ejercicio: Transformar el modelo conceptual que se expone a continuación al modelo lógico relacional.



4. DISEÑO FÍSICO

El diseño físico de datos define el esquema interno de la base de datos. Según Métrica v3 se define la estructura física de datos del Sistema de Información a partir del modelo lógico de datos normalizado.

En función del SGBD, los requisitos y el entorno, se busca la eficiencia, tratando de mejorar tiempos de respuesta y optimizar los recursos del sistema.

Por tanto, el objetivo es obtener un esquema interno de la BD que cumpla lo mejor posible los objetivos de funcionamiento de la BD que los usuarios esperan: minimizar el tiempo de respuesta de la BD y su espacio de almacenamiento e incrementar la seguridad.

El diseño físico se divide de cuatro fases, cada una de ellas compuesta por una serie de pasos:

- Traducir el esquema lógico para el SGBD específico.
- Diseñar la representación física.
- Diseñar los mecanismos de seguridad.
- Monitorizar y afinar el sistema.

1. Traducir el esquema lógico para el SGBD específico

Para ello, es necesario conocer toda la funcionalidad que el SGBD ofrece. Por ejemplo, el diseñador deberá saber:

- Si el sistema soporta la definición de claves primarias, claves ajenas y claves alternativas.
- Si el sistema soporta la definición de datos requeridos (es decir, si se pueden definir atributos como no nulos).
- Si el sistema soporta la definición de dominios.
- Si el sistema soporta la definición de restricciones o aserciones de usuario.
- Cómo se crean las tablas.

La tarea fundamental de construcción del esquema físico es la implementación de las relaciones o tablas obtenidas en el modelo lógico. Se definen mediante el **lenguaje de definición de datos (DDL)** del SGBD. Para ello, se utiliza la información producida durante el diseño lógico, es decir, el esquema lógico global.

El esquema lógico consta de un conjunto de relaciones o tablas y, para cada una de ellas, se tiene:

- El nombre de la relación.
- La lista de atributos entre paréntesis.
- La clave primaria y las claves ajenas, si las tiene.

- Las reglas de integridad de las claves ajenas.

Y para cada uno de los atributos se define:

- Su dominio: tipo de datos, longitud y restricciones de dominio.
- El valor por defecto, que es opcional.
- Si admite nulos.
- Si es derivado y, en caso de serlo, cómo se calcula su valor.

Como consecuencia de todo ello se podrán crear los “scripts” de la base de datos escritos con la sintaxis y funcionalidades del LDD del SGBDR. Las dos órdenes fundamentales a utilizar en este caso son: CREATE TABLE y CREATE DOMAIN.

Esta sintaxis, además, permite incluir referencias al modo de almacenamiento como pueden ser: ficheros físicos o lógicos (tablespaces) asignables a cada tabla, segmentos o bloque asignados inicialmente y en expectativa de crecimiento, particionamientos de la estructura, etc.

Además, será necesario completar la funcionalidad de las estructuras creadas mediante las denominadas **restricciones de usuario**, que permitirán delimitar las actualizaciones que se realizan sobre las relaciones de la base de datos mediante restricciones, aserciones o disparadores. En cuanto a las primeras, se definen mediante la cláusula de CREATE TABLE, CONSTRAINT CHECK (<condición>), las segundas mediante la orden CREATE ASSERTION y los últimos con CREATE TRIGGER.

2. Diseñar la representación física

Uno de los objetivos principales del diseño físico es almacenar los datos de modo eficiente. Para medir la eficiencia hay varios factores que se deben tener en cuenta:

- Productividad de transacciones. Es el número de transacciones que se quiere procesar por unidad de tiempo.
- Tiempo de respuesta. Es el tiempo que tarda en ejecutarse una transacción. Desde el punto de vista del usuario, este tiempo debería ser el mínimo posible.
- Espacio en disco. Es la cantidad de espacio en disco que hace falta para los ficheros de la base de datos. Normalmente, el diseñador deberá minimizar este espacio.

Normalmente, todos estos factores no se pueden satisfacer a la vez. Por lo tanto, el diseñador deberá ir ajustando estos factores para conseguir un equilibrio razonable. El diseño físico inicial no será el definitivo, sino que habrá que ir monitorizándolo para observar sus prestaciones e ir ajustándolo como sea oportuno.

ANALIZAR LAS TRANSACCIONES

Para realizar un buen diseño físico es necesario conocer las consultas y las transacciones que se van a ejecutar sobre la base de datos. Esto incluye tanto información cualitativa, como cuantitativa. Para cada transacción, hay que especificar:

- La frecuencia con que se va a ejecutar.
- Las relaciones y los atributos a los que accede la transacción, y el tipo de acceso: consulta, inserción, modificación o eliminación. Los atributos que se modifican no son buenos candidatos para construir estructuras de acceso.
- Los atributos que se utilizan en los predicados del WHERE de las sentencias SQL. Estos atributos pueden ser candidatos para construir estructuras de acceso, dependiendo del tipo de predicado que se utilice.
- Si es una consulta, los atributos involucrados en el JOIN de 2 o más relaciones. Estos atributos pueden ser candidatos para construir estructuras de acceso.
- Las restricciones temporales impuestas sobre la transacción. Los atributos utilizados en los predicados de la transacción pueden ser candidatos para construir estructuras de acceso.

ESCOGER LAS ORGANIZACIONES DE FICHEROS

En el caso de aquellos SGBDRs que permitan asociar cada estructura de datos con un fichero físico es recomendable escoger la organización de ficheros óptima para cada relación.

Por ejemplo, un fichero desordenado es una buena estructura cuando se va a cargar gran cantidad de datos en una relación al inicializarla, cuando la relación tiene pocas tuplas, también cuando en cada acceso se deben obtener todas las tuplas de la relación, o cuando la relación tiene una estructura de acceso adicional, como puede ser un índice.

Por otra parte, los ficheros dispersos (hashing) son apropiados cuando se accede a las tuplas a través de los valores exactos de alguno de sus campos (condición de igualdad en el WHERE).

Si la condición de búsqueda es distinta de la igualdad (búsqueda por rango, por patrón, etc.), la dispersión no es una buena opción. Hay otras organizaciones, como la ISAM o los árboles B+.

ESCOGER LOS ÍNDICES SECUNDARIOS

Los índices secundarios permiten especificar caminos de acceso adicionales para las relaciones base. Hay que tener en cuenta que estos índices conllevan un coste de mantenimiento que hay que sopesar frente a la ganancia en prestaciones. A la hora de seleccionar los índices, se pueden seguir las siguientes indicaciones:

- Construir un índice sobre la clave primaria de cada relación base, normalmente se crea de forma implícita con la cláusula CONSTRAINT PRIMARY KEY.
- No crear índices sobre tablas pequeñas.
- Añadir un índice sobre los atributos que se utilizan para acceder con mucha frecuencia.
- Añadir un índice sobre las claves ajenas que se utilicen con frecuencia para hacer Joins.

- Evitar los índices sobre atributos que se modifican a menudo.
- Evitar los índices sobre atributos poco selectivos (aquellos en los que la consulta selecciona una porción significativa de la tabla).
- Evitar los índices sobre atributos formados por cadenas de caracteres largas.

CONSIDERAR LA INTRODUCCIÓN DE REDUNDANCIAS CONTROLADAS

En ocasiones, puede ser conveniente relajar las reglas de normalización introduciendo redundancias de forma controlada, con objeto de mejorar las prestaciones del sistema. En la etapa del diseño lógico se recomienda llegar, al menos, hasta la tercera forma normal para obtener un esquema con una estructura consistente y sin redundancias. Pero, a menudo, sucede que las bases de datos, así normalizadas, no proporcionan la máxima eficiencia, con lo que es necesario volver atrás y desnormalizar algunas relaciones. Es importante hacer notar que la desnormalización sólo debe realizarse cuando se estime que el sistema no puede alcanzar las prestaciones deseadas. Por lo tanto, hay que tener en cuenta los siguientes factores:

- La desnormalización hace que la implementación sea más compleja.
- La desnormalización hace que se sacrifique la flexibilidad.
- La desnormalización puede hacer que los accesos a datos sean más rápidos, pero ralentiza las actualizaciones.

Optimización

La optimización consiste en una **desnormalización controlada del modelo físico de datos** que se aplica para reducir o simplificar el número de accesos a la base de datos.

El objetivo de esta técnica es reestructurar el modelo físico de datos con el fin de asegurar que satisface los requisitos de rendimiento establecidos y conseguir una adecuada eficiencia del sistema.

Para ello, se seguirán alguna de las **recomendaciones** que a continuación se indican:

- Introducir elementos redundantes.
- Dividir entidades.
- Combinar entidades si los accesos son frecuentes dentro de la misma transacción.
- Redefinir o añadir relaciones entre entidades para hacer más directo el acceso entre entidades.
- Definir claves secundarias o índices para permitir caminos de acceso alternativos.

Con el fin de analizar la conveniencia o no de la desnormalización, se han de considerar, entre otros, los siguientes aspectos:

- Los tiempos de respuesta requeridos.
- La tasa de actualizaciones respecto a la de recuperaciones.
- Las veces que se accede conjuntamente a los atributos.

- La longitud de los mismos.
- El tipo de aplicaciones (en línea / por lotes).
- La frecuencia y tipo de acceso.
- La prioridad de los accesos.
- El tamaño de las tablas.
- Los requisitos de seguridad: accesibilidad, confidencialidad, integridad y disponibilidad que se consideren relevantes.

3. Diseñar los mecanismos de seguridad

Los datos constituyen un recurso esencial para la organización, por lo tanto su seguridad es de vital importancia. Durante el diseño lógico se habrán especificado los requerimientos en cuanto a seguridad que en esta fase se deben implementar. Para llevar a cabo esta implementación, el diseñador debe conocer las posibilidades que ofrece el SGBD que se vaya a utilizar.

- **Diseñar las vistas de los usuarios:** las vistas de los usuarios se corresponden a los esquemas lógicos locales. Las vistas, además de preservar la seguridad, mejoran la independencia de datos, reducen la complejidad y permiten que los usuarios vean los datos en el formato deseado. Para crearlas se utiliza la orden *CREATE VIEW*.
- **Diseñar las reglas de acceso:** para cada usuario o grupo de usuarios se determinan tanto los permisos sobre determinados objetos de la base de datos, como los permisos o privilegios del sistema, es decir, las operaciones de DDL que están disponibles para ese usuario o grupo. Normalmente, los privilegios se agrupan en conjuntos denominados «Roles». Para otorgar un permiso o rol se utiliza la orden *GRANT* y para denegarlo *REVOKE*. Se definen mediante el **Lenguaje de Control de Datos (DCL)**.

4. Monitorizar y afinar el sistema

Una vez implementado el esquema físico de la base de datos, se debe poner en marcha para observar su rendimiento (monitorizar). En el caso de que no se satisfagan las necesidades de rendimiento deseadas, el esquema deberá ser modificado (afinar).

Este proceso no es estático y puntual, sino que la monitorización está presente a lo largo de toda la vida del sistema, requiriéndose nuevas modificaciones (refinar) para adaptarse a los nuevos requisitos.

5. EL MODELO LÓGICO RELACIONAL

El modelo relacional es un modelo con sólidos fundamentos matemáticos, basado en la teoría de conjuntos. Fue definido por E. F. Codd en 1970.

Las características fundamentales del modelo relacional son:

- Las **estructuras de datos son simples**: se trata de relaciones que se presentan al usuario en forma de tablas bidimensionales, permitiendo un alto grado de independencia de la información, con respecto al medio físico de almacenamiento de los datos.
- Proporciona una base sólida para la consistencia de los datos a través de las **reglas de integridad**. Igualmente, el proceso de normalización, al eliminar ciertas anomalías que pueden presentarse en las relaciones, representa una valiosa ayuda para el diseño de la BD.
- Permite la manipulación de las relaciones en forma **orientada a conjuntos**. Esto ha conducido al desarrollo de lenguajes muy potentes basados, bien en la teoría de conjuntos (álgebra relacional), bien en la lógica de predicados (cálculo relacional).

1. Conceptos fundamentales del modelo relacional

El esquema de una BDR se compone de uno o más esquemas de relación y de un conjunto de restricciones de integridad.

Un esquema de relación consiste en el nombre de relación, seguido de los nombres de los atributos:

Nombre_Relación (Atributo1, Atributo2, ..., AtributoN)

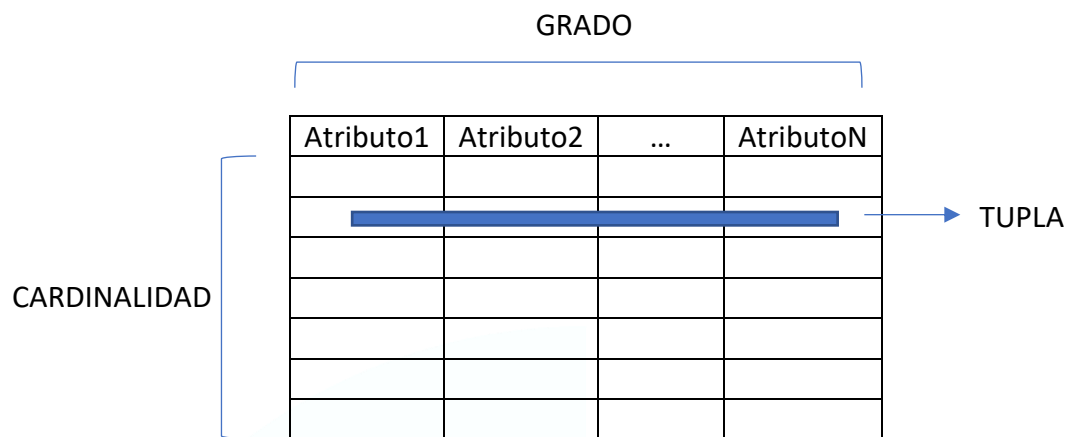
La definición formal de una relación: “*Dados los dominios $D_1, D_2, \dots, D_n$ (no necesariamente distintos), R es una relación entre estos n conjuntos si es un conjunto de n tuplas $(d_1, d_2, \dots, d_n)$ tal que d_1 pertenece a $D_1 \dots d_n$ pertenece a D_n* ”.

Un dominio es un conjunto de valores.

Cada atributo, o propiedad con interés informativo de una relación está asociado a un dominio del que toma sus posibles valores.

El número de atributos de una relación define su **grado**, mientras que el número de tuplas de la relación define su **cardinalidad**.

La extensión u ocurrencia de una relación es una tabla donde las filas corresponden a las **tuplas** y las columnas a los **atributos**, es decir, es el conjunto de tuplas de una relación.



De estas definiciones se deducen las características de una tabla de estructura relacional:

- Cada tabla debe contener un solo tipo de filas. El formato de cada fila queda definido por el esquema de la relación. Es decir, todas las filas tienen las mismas columnas y formato.
- Cada fila tiene que ser única, no puede haber filas duplicadas.
- El orden de las filas dentro de una tabla es indiferente.
- Cada columna debe estar identificada por un nombre específico.
- El orden de las columnas dentro de una tabla es indiferente.
- Cada columna debe extraer sus valores de un dominio.
- Un mismo dominio podrá servir para definir los valores de varias columnas diferentes.
- El valor individual de la intersección de cualquier fila y columna será un único dato.

2. Reglas de integridad

Permiten definir propiedades de los datos que no pueden ser capturadas basándose en conjuntos y relaciones. Las razones para que un modelo requiera restricciones son:

- **Semántica:** permite reflejar más exactamente la información a modelar.
- **Integridad:** comunica al SGBD qué estados de la BD están permitidos.

Al definir cada atributo sobre un dominio se impone una restricción sobre el conjunto de valores permitidos para cada atributo. A este tipo de restricciones se les denomina restricciones de dominios.

Hay además dos reglas de integridad muy importantes que son restricciones que se deben cumplir en todas las bases de datos relacionales y en todos sus estados o instancias (las reglas se deben cumplir todo el tiempo). Estas reglas son la **regla de integridad de entidades** y la **regla de integridad referencial**.

Antes de definir las, es preciso conocer el concepto de **nulo**. Cuando en una tupla un atributo es **desconocido**, se dice que es nulo. Un nulo no representa el valor cero ni la cadena vacía, éstos son valores que tienen significado. El nulo implica ausencia de información, bien porque al insertar la tupla se desconocía el valor del atributo, o bien porque para dicha tupla el atributo no tiene sentido.

Regla de integridad de la ENTIDAD

Se aplica a las **claves primarias** de las relaciones. Ninguno de los atributos que componen la clave primaria puede ser nulo.

Por definición, una clave primaria es un identificador irreducible que se utiliza para identificar de modo único las tuplas. Que sea irreducible significa que ningún subconjunto de la clave primaria sirve para identificar las tuplas de modo único. Si se permite que parte de la clave primaria sea nula, se está diciendo que no todos sus atributos son necesarios para distinguir las tuplas, con lo que se contradice la irreducibilidad.

Esta regla sólo se aplica a las relaciones y a las claves primarias, no a las claves alternativas.

Regla de integridad REFERENCIAL

La segunda regla de integridad se aplica a las **claves ajenas**. Si en una relación hay alguna clave ajena, sus valores deben coincidir con valores de la clave primaria a la que hace referencia, o bien, deben ser completamente nulos.

Ejemplo: clave ajena de la relación libro que referencia a una editorial.

Editorial(**cod_editorial**, direccion, provincia, ...)

Libro(**cod libro**, titulo, ..., **cod_editorial**)

Ejemplo: claves ajenas de la relación Escribe que referencia a un Autor y a un Libro respectivamente.

Autor(**DNI**, nombre, apellidos, ...)

Libro(**cod libro**, titulo, ..., cod_editorial)

Escribe(**autor**, **cod libro**, fecha, otros atributos, ...)

3. Lenguajes relaciones

Son varios los lenguajes utilizados por los SGBD relacionales para manejar las relaciones. Algunos de ellos son procedurales, lo que quiere decir que el usuario dice al sistema exactamente cómo debe manipular los datos. Otros son no procedurales, que significa que el usuario dice qué datos necesita, en lugar de decir cómo deben obtenerse.

La base de los lenguajes relacionales la constituyen el **álgebra relacional** y el **cálculo relacional**. Ambos lenguajes son equivalentes: para cada expresión del álgebra, se puede encontrar una expresión equivalente en el cálculo, y viceversa.

El álgebra relacional (o el cálculo relacional) se utiliza para medir la potencia de los lenguajes relacionales.

Si un lenguaje permite obtener cualquier relación que se pueda derivar mediante el álgebra relacional, se dice que es relacionalmente completo. La mayoría de los lenguajes relacionales son relacionalmente completos, pero tienen más potencia que el álgebra o el cálculo porque se les han añadido operadores especiales. En este apartado se verán las características más importantes correspondientes al álgebra relacional.

Álgebra relacional

Lenguaje formal con una serie de operadores que trabajan sobre una o varias relaciones para obtener otra relación resultado, sin que cambien las relaciones originales.

Tanto los operandos como los resultados son relaciones, por lo que la salida de una operación puede ser la entrada de otra operación. Esto permite anidar expresiones del álgebra, del mismo modo que se pueden anidar las expresiones aritméticas.

De los ocho operadores, solo hay 5 fundamentales, que forman un *conjunto relacionalmente completo*, es decir, permite obtener cualquier subconjunto de los datos contenidos en una BD:

- **Selección:** opera sobre una sola relación R y da como resultado otra relación cuyas tuplas son las tuplas de R que satisfacen la condición especificada (C). Esta condición es una comparación en la que aparece al menos un atributo de R, o una combinación booleana de varias de estas comparaciones.

$\sigma_C(R)$

R1	
CÓDIGO	PROVINCIA
01	Álava
02	Albacete
03	Alicante
28	Madrid

Selección
«F» = (Código = «01») OR
(Provincia > «Alicante»)

$\sigma_F(R1)$	
CÓDIGO	PROVINCIA
01	Álava
28	Madrid

- **Proyección:** opera sobre una sola relación R y da como resultado otra relación que contiene un subconjunto vertical de R, extrayendo los valores de los atributos especificados y eliminando duplicados.

$$\pi_{1..n}(R)$$

R1	
CÓDIGO	PROVINCIA
01	Álava
02	Albacete
03	Alicante
28	Madrid

Proyección
«x» = provincia

$\pi_x(R1)$	
PROVINCIA	
Álava	
Albacete	
Alicante	
Madrid	

- **Producto cartesiano:** relación resultante de combinar cada fila de la relación R con todas las de la relación S. Ya que es posible que haya atributos con el mismo nombre en las dos relaciones, el nombre de la relación se antepone al del atributo, en este caso, para que los nombres de los atributos sigan siendo únicos en la relación resultado. En la relación resultante:
 - o El grado es la suma del número de columnas de R y S.
 - o La cardinalidad es el producto del número de filas de R por el número de filas de S.

$$R \times S$$

R1		R2	R1 × R2		
CÓDIGO	PROVINCIA	CANTIDAD	CÓDIGO	PROVINCIA	CANTIDAD
01	Álava	100	01	Álava	100
02	Albacete	500	01	Álava	500
03	Alicante		02	Albacete	100
28	Madrid		02	Albacete	500
			03	Alicante	100
			03	Alicante	500
			28	Madrid	100
			28	Madrid	500

- **Unión:** dadas dos relaciones R y S, se define la unión entre ambas como la relación resultante de tomar las filas que están en una u otra relación y además las que están en ambas (sólo una vez).

Para que se pueda producir la unión es necesario que las relaciones R y S presenten igual grado y compatibilidad de dominios para cada par de atributos tomados de uno en uno entre ambas relaciones, es decir, r_1 ha de ser de un dominio igual o compatible a s_1 y, en general, r_N ha de ser de un dominio igual o compatible a s_N .

RuS

R1		R2		R1 ∪ R2	
CÓDIGO	PROVINCIA	CÓDIGO	PROVINCIA	CÓDIGO	PROVINCIA
01	Álava	01	Álava	01	Álava
02	Albacete	08	Barcelona	02	Albacete
03	Alicante			03	Alicante
28	Madrid			08	Barcelona
				28	Madrid

- **Diferencia:** dadas dos relaciones R y S, se define la diferencia entre ambas como la relación resultante de tomar las filas que están en R y no están en S. Se trata de una operación no conmutativa, a diferencia de las anteriores.

R-S

R1		R2		R1 - R2	
CÓDIGO	PROVINCIA	CÓDIGO	PROVINCIA	CÓDIGO	PROVINCIA
01	Álava	01	Álava		
02	Albacete	08	Barcelona	02	Albacete
03	Alicante			03	Alicante
28	Madrid			28	Madrid

Operadores NO fundamentales:

- **Join:** dadas dos relaciones R y S, se define la concatenación o join como la relación resultante del producto cartesiano entre ambas tablas una vez seleccionadas aquellas filas que tomen igual valor en aquella/s filas expresadas en la operación. Para que la operación se pueda producir es necesario que ambas relaciones presenten, al menos, un campo en común sobre el que establecer la condición de igualdad

R|x|S

R1	A	B	C	R2	B	C	D
	1	2	3		5	6	9
	4	5	6		3	3	8
	7	8	9		2	3	3

R1	A	B _{R1}	C _{R1}	B _{R2}	C _{R2}	D
	1	2	3	5	6	9
	1	2	3	3	3	8
	1	2	3	2	3	3
	4	5	6	5	6	9
	4	5	6	3	3	8
	4	5	6	2	3	3
	7	8	9	5	6	9
	7	8	9	3	3	8
	7	8	9	2	3	3

σ	A	B _{R1}	C _{R1}	B _{R2}	C _{R2}	D
	1	2	3	2	3	3
	4	5	6	5	6	9

R1 $\times$ R2	A	B	C	D
	1	2	3	3
	4	5	6	9

- **Intersección:** dadas 2 relaciones R y S, se define la intersección como la relación resultante de tomar las filas que están tanto en R como en S. Al igual que en la unión, las relaciones R y S han de presentar igual grado y compatibilidad de dominios para cada par de atributos tomados de uno en uno entre ambas relaciones.

$R \cap S$

R1		R2		R1 $\cap$ R2	
CÓDIGO	PROVINCIA	CÓDIGO	PROVINCIA	CÓDIGO	PROVINCIA
01	Álava	01	Álava	01	Álava
02	Albacete	08	Barcelona		
03	Alicante	28	Madrid		
28	Madrid			28	Madrid

- **División:** dadas 2 relaciones R y S en las que existe un subconjunto de atributos (X) que están formando parte de S y R al mismo tiempo y otro conjunto de atributos (Y) que únicamente forman parte de R, se define la división o cociente como la relación resultante de combinar cada fila de Y con todas las filas que forman parte de los atributos X. En definitiva, para que t aparezca en el resultado, los valores de t deben aparecer en R en combinación con cada tupla de S.

$R \div S$

A	B
a1	b1
a1	b2
a2	b1
a3	b2

 $\div$

B
b1
b2

 $=$

A
a1

6. NORMALIZACIÓN

La teoría de la normalización, como técnica formal para organizar los datos, ayuda a encontrar fallos y a corregirlos, **evitando así introducir anomalías en las operaciones de manipulación de datos**.

Se dice que:

“Una relación está en una determinada forma normal si satisface un cierto conjunto de restricciones sobre los atributos.”

Cuanto más restricciones existan, menor será el número de relaciones que las satisfagan, así, por ejemplo, una relación en tercera forma normal estará también en segunda y en primera forma normal. Y, en consecuencia, cuanto más alta sea la forma normal aplicable a una relación menos vulnerable será a inconsistencias y anomalías.

La teoría de la normalización tiene por objetivo:

- La **eliminación de dependencias entre atributos** que originen anomalías en la actualización de los datos.
- Proporcionar una **estructura más regular para la representación de las tablas**, constituyendo el soporte para el diseño de bases de datos relacionales.

Como resultado de la aplicación de esta técnica se obtiene un modelo lógico de datos normalizado.

Antes de definir las distintas formas normales se explican, muy brevemente, algunos conceptos necesarios para su comprensión.

Dependencia funcional

Un atributo Y se dice que depende funcionalmente de otro X si, y sólo si, a cada valor de X le corresponde un único valor de Y, lo que se expresa de la siguiente forma: $X \rightarrow Y$ (también se dice que X determina o implica a Y). X se denomina implicante o determinante e Y es el implicado.

Dependencia funcional completa

Un atributo Y tiene dependencia funcional completa respecto de otro X, si depende funcionalmente de él en su totalidad, es decir, no depende de ninguno de los posibles atributos que formen parte de X.

Dependencia transitiva

Un atributo depende transitivamente de otro si, y sólo si, depende de él a través de otro atributo. Así, Z depende transitivamente de X, si:

$X \rightarrow Y$

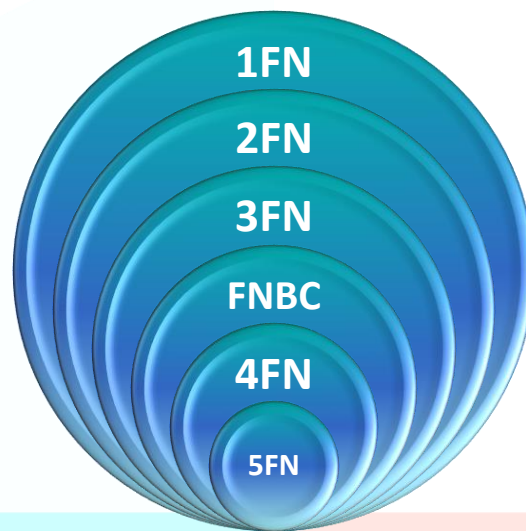
$Y \twoheadrightarrow X$

$Y \rightarrow Z$

Se dice que X implica a Z a través de Y.

Una vez definidas las anteriores dependencias, se pueden enunciar las formas normales:

- **Primera, Segunda y Tercera Formas Normales:** Codd en 1970.
- **Forma Normal de Boyce y Codd (FNBC):** Boyce-Codd en 1974.
- **Cuarta Forma Normal (4FN):** Fagin en 1977.
- **Quinta Forma Normal (5FN):** Fagin en 1979.



1. Primera Forma normal (1FN)

Una relación está en 1FN **si no tiene grupos repetitivos**, es decir, un atributo sólo puede tomar un único valor de un dominio simple.

Una vez identificados los atributos que no dependen funcionalmente de la clave principal, se formará con ellos una nueva relación y se eliminarán de la antigua. La clave principal de la nueva relación estará formada por la concatenación de uno o varios de sus atributos más la clave principal de la antigua relación.

Ejemplo:

R (DNI_P, Nombre, Teléfono)

DNI_P	NOMBRE	TELÉFONO
5553344	ZUTANO	99 5553322 99 4433221
6662266	MENGANO	99 3223378
...	...	...
...	...	...

NO ESTÁ EN 1FN

DNI_P	NOMBRE	TELÉFONO
5553344	ZUTANO	99 5553322
5553344	ZUTANO	99 4433221
6662266	MENGANO	99 3223378
...	...	...

ESTÁ EN 1FN

Se soluciona repitiendo toda la tupla para cada uno de los valores del grupo repetitivo.

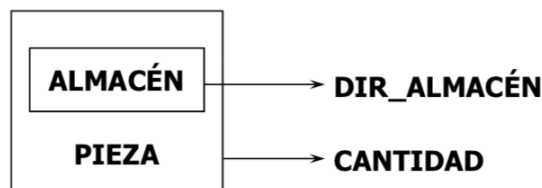
2. Segunda Forma normal (2FN)

Una relación está en 2FN si está en 1FN y **todos los atributos que no forman parte de las claves candidatas (atributos no principales) tienen dependencia funcional completa respecto de éstas**, es decir, no hay dependencias funcionales de atributos no principales respecto de una parte de las claves. Cada uno de los atributos de una relación depende de toda la clave.

Una vez identificados los atributos que no dependen funcionalmente de toda la clave, sino sólo de parte de la misma, se formará con ellos una nueva relación y se eliminarán de la antigua. La clave principal de la nueva relación estará formada por la parte de la antigua de la que dependen funcionalmente.

Ejemplo:

R (PIEZA, ALMACÉN, CANTIDAD, DIR_ALMACÉN)



¡NO ESTÁ EN 2 FN!

Solución:

R1(ALMACÉN, PIEZA, CANTIDAD)

R2(ALMACÉN, DIR_ALMACÉN)

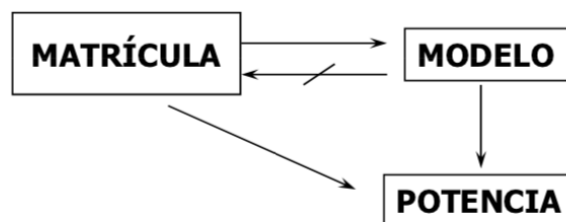
3. Tercera Forma normal (3FN)

Una relación está en 3FN si está en 2FN y **todos sus atributos no principales dependen directamente de alguna de las claves**, es decir, no hay dependencias funcionales transitivas de atributos no principales respecto de las claves.

Una vez identificados los atributos que dependen de otro atributo distinto de la clave, se formará con ellos una nueva relación y se eliminarán de la antigua. La clave principal de la nueva relación será el atributo del cual dependen. Este atributo en la relación antigua pasará a ser una clave ajena.

Ejemplo:

R (MATRÍCULA, MODELO, POTENCIA)



¡NO ESTÁ EN 3 FN!

Solución:

R1(MATRÍCULA, MODELO)

R2(MODELO, POTENCIA)

4. Forma normal de Boyce-Codd (FNBC)

Una relación está en Forma Normal de Boyce-Codd si y sólo si está en 3FN y además **todo determinante es una clave candidata**.

Está a medio camino entre la 3FN -ligeramente más fuerte- pero todavía no es 4FN.

Ejemplo:

R(dni, nombre, codalumno, codasig, nota)

Claves candidatas: (dni, codasig) y (codalumno, codasig) y se cumple que dni → nombre

dni es un determinante que no es clave candidata y, por tanto, **R no está en FNBC**.

Solución:

R1(dni, codalumno, nombre)

R2(dni, codasig, nota)

Ejemplo:

R(alumno, asignatura, profesor, nota)

Dependencias:

{alumno, asignatura} → profesor

{alumno, asignatura} → nota

{profesor} → asignatura

Claves candidatas:

{alumno, asignatura}

{alumno, profesor}

{profesor} → asignatura y, por tanto, **R no está en FNBC** ya que Profesor no es clave candidata.

Solución:

R1(alumno, asignatura, nota)

R2(profesor, asignatura)

5. Cuarta Forma normal (4FN)

Una relación está en cuarta forma normal si y sólo si está en FNBC y además **en todas las dependencias múltiplemente valoradas el implicante es clave candidata**.

Ejemplo: Un valor de X puede tener varios valores de Y.

Empleado(Nombre, Proyecto, Hijo)

Nombre	Proyecto	Hijo
Príamo	Caballo	Cassandra
Príamo	Troya	Paris
Príamo	Caballo	Paris
Príamo	Troya	Cassandra

Nombre →→ Proyecto | Hijo y, por tanto, **R no está en 4FN**.

Solución:

Nombre	Proyecto
Príamo	Caballo
Príamo	Troya

Nombre	Hijo
Príamo	Cassandra
Príamo	Paris

6. Quinta Forma normal (5FN)

Una relación está en quinta forma normal si y sólo si está en 4FN y además **toda dependencia de combinación** está implicada por una clave candidata.

Una dependencia de combinación aparece en relaciones que no pueden descomponerse en otras dos sin perder información. Para evitar esta pérdida de información hay que descomponerla, al menos, en otras tres relaciones.

Si en cada dependencia de reunión denotada por $(X_1, X_2, \dots, X_n)$, cada X_i es una clave candidata.

Cuando una relación se descompone en más de dos relaciones (porque no se pueda encontrar una descomposición sin pérdidas en dos proyecciones), se ha de cumplir este requisito para para que la descomposición sea sin pérdidas.

Ejemplo:

Solidarios(ONG, Proyecto, Lugar)

(WWF/Adena, Árboles, Mundo)

(GreenPeace, Árboles, España)

(WWF/Adena, Aire, España)

Solución: dividir R en R1, R2, ..., Rn

R1(ONG, Proyecto)

R2(Proyecto, Lugar)

R3(ONG, Lugar)

Si aplicamos una Reunión Natural a dos de esas tres relaciones, se producen tuplas falsas, pero si aplicamos una Reunión Natural a las tres NO se producen tuplas falsas.

	ONG	Proyecto	Lugar	
→	WWF/Adena	Árboles	Mundo	Proyección
→	GreenPeace	Árboles	España	→
→	WWF/Adena	Aire	España	←
→	WWF/Adena	Árboles	España	Reunión
	WWF/Adena	Biodiversidad	Mundo	
	Ing. Sin Fronteras	Desarrollo	Mundo	

ONG	Proyecto
WWF/Adena	Árboles
GreenPeace	Árboles
WWF/Adena	Aire
WWF/Adena	Biodiversidad
Ing. Sin Fronteras	Desarrollo

Proyecto	Lugar
Árboles	Mundo
Árboles	España
Aire	España
Biodiversidad	Mundo
Desarrollo	Mundo

ONG	Lugar
WWF/Adena	Mundo
GreenPeace	España
WWF/Adena	España
Ing. Sin Fronteras	Mundo